# PROGRAMMING IN JAVA

## Assignment 5

### TYPE OF QUESTION: MCQ

**Number of questions**: 10 Total marks**: 10 × 1 = 10**

---

## QUESTION 1:

**Which keyword is NOT used by Java during exception handling?**

a. **try**

b. **catch**

c. **final**

d. **finally**

**Correct Answer: c**

c. **final**

**Detailed Solution:**

In Java, exceptions are handled using the `try`, `catch`, and `finally` blocks. The `try` block contains code that might throw an exception, the `catch` block handles specific exceptions, and the `finally` block executes regardless of whether an exception occurs.

---

## QUESTION 2:

**What is the purpose of the `throws` keyword in Java?**

a. **To declare exceptions that a method can throw**

b. **To throw an exception immediately**

c. **To catch an exception**

d. **It is not a keyword in Java**

**Correct Answer: a**

a. **To declare exceptions that a method can throw**

**Detailed Solution:**

The `throws` keyword is used in a method signature to declare the exceptions that the method might throw, alerting callers to handle or propagate these exceptions.

## QUESTION 3:

**Which of the following is TRUE about interfaces in Java?**

a. **Interfaces should always be defined as final**

b. **Interfaces can be instantiated directly.**

c. **Interfaces can extend multiple interfaces.**

d. **Interfaces cannot have any method signatures**

**Correct Answer: c**

c. **Interfaces can extend multiple interfaces.**

**Detailed Solution:**

In Java, an interface can extend multiple interfaces. Interfaces can also contain `public`, `static`, and `final` variables, but they cannot be instantiated directly.

## QUESTION 4:

**What will be the output of the following code?**

```
interface Demo {
    void display();
}

class Test implements Demo {
    public void display() {
        System.out.println("Hello, NPTEL!");
    }
}

public class Main {
    public static void main(String[] args) {
        Test obj = new Test();
        obj.display();
    }
}
```

a. **Hello, NPTEL!**

b. **Compilation Error**

c. **Runtime Error**

d. **No Output**

**Correct Answer: a**

a. **Hello, NPTEL!**

**Detailed Solution:**

The `Test` class implements the `Demo` interface and provides a definition for the `display` method. When `display()` is called, it prints “Hello, NPTEL!”.

## QUESTION 5:

**What will be the output of the following Java program?**

```
interface Calculator {
    void calculate(int value);
}

class Square implements Calculator {
    int result;
    public void calculate(int value) {
        result = value * value;
        System.out.print("Square: " + result + " ");
    }
}

class Cube extends Square {
    public void calculate(int value) {
        result = value * value * value;
        super.calculate(value);
        System.out.print("Cube: " + result + " ");
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator obj = new Cube();
        obj.calculate(3);
    }
}
```

a. **Square: 9 Cube: 9**

b. **Cube: 27 Square: 9**

c. **Square: 9 Square: 27 Cube: 27**

d. **Square: 9 Cube: 27 Square: 27**

**Correct Answer: a**

a. **Square: 9 Cube: 9**

**Detailed Solution:**

The `Cube` class overrides the `calculate` method of the `Square` class. In the `Cube` class's `calculate` method, `super.calculate(value)` is called, which executes the `calculate` method of the `Square`

class. First, "Square: 9" is printed by the superclass method. The call to super.calculate(value) overwrites the previously computed cube value. Then, the overridden method in `Cube` prints "Cube: 9".

## QUESTION 6:

**What will be the output of the following program?**

```
class NPTEL {
    public static void main(String[] args) {
        try {
            int a = 5;
            int b = 0;
            System.out.println(a / b);
        } catch (ArithmeticException e) {
            System.out.print("Error ");
        } finally {
            System.out.print("Complete");
        }
    }
}
```

a. **5 Complete**

b. **Error Complete**

c. **Runtime Error**

d. **Compilation Error**

**Correct Answer: b**

b. **Error Complete**

**Detailed Solution:**

An `ArithmeticException` is caught in the `catch` block, which prints “Error”. The `finally` block executes afterward, printing “Complete”.

## QUESTION 7:

**What will be the output of the following Java program?**

```
interface A {
    int x = 10;
    void display();
}
class B implements A {
    public void display() {
        System.out.println("Value of x: " + x);
    }
}
public class Main {
    public static void main(String[] args) {
        B obj = new B();
        obj.display();
    }
}
```

a. **Value of x: 10**

b. **Value of x: 0**

c. **Compilation Error**

d. **Runtime Error**

**Correct Answer: a**

a. **Value of x: 10**

**Detailed Solution:**

Variables in interfaces are `public`, `static`, and `final` by default. Hence, the value of `x` is accessible in the `display` method.

## QUESTION 8:

**Which keyword is used by Java during exception handling?**

a. **final**

b. **finally**

c. **caught**

d. **finished**

**Correct Answer: b**

b. **finally**

**Detailed Solution:**

In Java, exceptions are handled using the try, catch, and finally blocks. The try block contains code that might throw an exception, the catch block handles specific exceptions, and the finally block executes regardless of whether an exception occurs.

## QUESTION 9:

**A method that potentially generates a checked exception must include this keyword in its method signature:**

a. **throw**

b. **extend**

c. **throws**

d. **extends**

**Correct Answer: c**

c. **throws**

**Detailed Solution:**

Any Java class that generates a checked exception and does not handle it internally must use the throws keyword to alert other methods of its instability.

## QUESTION 10:

**Consider the following code.**

```
class Test extends Exception {}

class Main {

  public static void main(String args[]) {
    try {
      throw new Test();
    } catch (Test t) {
      System.out.println("Got the Test Exception");
    } finally {
      System.out.println("Inside finally block ");
    }
  }
}
```

**What will be the output of the above code?**

a. **Got the Test Exception**
   **Inside finally block**

b. **Got the Test Exception**

c. **Inside finally block**

d. **Compiler Error**

**Correct Answer: a**

a. **Got the Test Exception**
   **Inside finally block**

**Detailed Solution:**

In Java, the finally is always executed after the try-catch block. This block can be used to do the common cleanup work.